

Italian Electricity Demand Forecasting — Project Report

1. Executive Summary

This project demonstrates an end-to-end machine learning pipeline for **short-term electricity demand forecasting** on the Italian power grid (Terna/ENTSO-E data). The system covers:

- **Data ingestion** from ENTSO-E Transparency Platform with realistic synthetic fallback
 - **Feature engineering** (54 temporal, lag, rolling, and cyclical features)
 - **Forecasting** with XGBoost and LightGBM ($R^2 > 0.94$, MAPE $\sim 4.4\%$)
 - **Anomaly detection** using Isolation Forest, Z-score, and IQR methods
 - **Classification** of peak vs off-peak demand periods (AUC > 0.98)
 - **Explainability** via SHAP values (global and per-instance)
 - **MLOps** with MLflow experiment tracking
 - **Automated reporting** with optional LLM enhancement
-

2. Motivation

Electricity demand forecasting is critical for: - **Grid stability**: Operators must balance supply and demand in real-time - **Energy trading**: Accurate forecasts reduce procurement costs - **Infrastructure planning**: Peak demand determines grid capacity requirements - **Renewable integration**: Variable renewables (wind, solar) require accurate demand baselines

Italy's electricity demand follows strong temporal patterns driven by industrial activity, heating/cooling loads, and solar generation. This makes it an ideal case study for gradient boosting methods.

3. Data

3.1 Source

ENTSO-E Transparency Platform (European Network of Transmission System Operators for Electricity): - Real-time and historical data for all European electricity markets - Hourly resolution, freely accessible via REST API - Data fields: TotalLoadValue (MW), generation by source, grid frequency

API endpoint: <https://web-api.tp.entsoe.eu/api>

Area code for Italy: 10YIT-GRTN-----B

3.2 Synthetic Data

When the API key is pending or unavailable, the pipeline generates realistic synthetic data: - **Demand**: 3 years (2022-2024) of hourly demand with daily cycles, weekly patterns, seasonal trends, and noise - **Generation**: 4 sources (thermal, solar, wind, hydro) with physical constraints - **Frequency**: Grid frequency around 50 Hz nominal with realistic fluctuations

The synthetic data preserves real statistical properties (autocorrelation, distribution shape) making the pipeline fully functional without API access.

3.3 Data Characteristics

Property	Value
Resolution	Hourly
Period	2022-01-01 to 2024-12-31
Total samples	26,281
Mean demand	~25,000 MW
Peak demand	~40,000 MW
Min demand	~15,000 MW

4. Feature Engineering

The feature engineering pipeline (`src/data/feature_engineering.py`) transforms raw hourly demand into 54 predictive features:

4.1 Temporal Features (11)

- `hour`, `dayofweek`, `dayofyear`, `month`, `year`, `week`, `quarter`
- `is_weekend` (binary), `is_night` (binary), `is_peak` (binary, top 25%)
- **Cyclical encodings:** `hour_sin`, `hour_cos`, `month_sin`, `month_cos`, `dayofweek_sin`, `dayofweek_cos`

The cyclical encoding is critical: hour 23 is close to hour 0, but a naive integer encoding would make them far apart. Sin/cos transforms preserve this circular relationship.

4.2 Lag Features (8)

- Lagged demand values at 1, 2, 3, 6, 12, 24, 48, 168 hours
- Lag-24 captures the “same time yesterday” pattern
- Lag-168 captures the “same time last week” pattern

4.3 Rolling Statistics (20)

- Rolling mean, std, min, max at windows: 6, 12, 24, 48, 168 hours
- These capture local trends and volatility

4.4 Exponentially Weighted Moving Averages (3)

- EWM with spans 12, 24, 168 hours
- More weight on recent observations

4.5 Difference Features (3)

- First difference at lags 1, 24, 168
- Captures rate of change

4.6 Train/Test Split

Chronological split (not random): 80% train, 20% test - Train: 2022-01-01 to 2024-03-29 (~20,890 hours) - Test: 2024-03-29 to 2024-12-31 (~5,223 hours)

Random splitting would leak future information into training — always use chronological splits for time series.

5. Methods

5.1 Gradient Boosting Forecasting

Both XGBoost and LightGBM are tree-based ensemble methods that build trees sequentially, with each new tree correcting the errors of the previous ones.

XGBoost (Extreme Gradient Boosting)

```
n_estimators: 500
max_depth: 6
learning_rate: 0.05
subsample: 0.8
colsample_bytree: 0.8
min_child_weight: 5
reg_alpha: 0.1 (L1 regularization)
reg_lambda: 1.0 (L2 regularization)
```

LightGBM (Light Gradient Boosting)

```
n_estimators: 500
max_depth: 6
learning_rate: 0.05
subsample: 0.8
colsample_bytree: 0.8
min_child_samples: 20
reg_alpha: 0.1
reg_lambda: 1.0
```

Why gradient boosting for this task? 1. Handles non-linear relationships naturally (demand vs hour is not linear) 2. Robust to outliers (tree-based splits) 3. No feature scaling required 4. Built-in feature importance 5. Fast training on tabular data 6. State-of-the-art for short-term load forecasting in literature

5.2 Anomaly Detection

Three complementary methods provide robust anomaly identification:

Isolation Forest

- **Principle:** Anomalies are “few and different” — they are easier to isolate (fewer splits needed)
- **Parameters:** contamination=0.01 (expect 1% anomalies), n_estimators=200

- **Output:** Anomaly score (negative = more anomalous)
- **Advantage:** Multivariate — considers the full feature context

Z-Score Method

- **Principle:** Points > 3 standard deviations from the mean are anomalous
- **Parameters:** threshold=3.0
- **Output:** Binary anomaly flag + z-score
- **Advantage:** Simple, interpretable, no training required

IQR (Interquartile Range)

- **Principle:** Values outside $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ are outliers
- **Parameters:** multiplier=1.5
- **Output:** Binary flag + bounds
- **Advantage:** Robust to non-normal distributions

Anomaly Type Classification

- **Spike:** Rolling 24h z-score > 2 (demand unexpectedly high)
- **Dip:** Rolling 24h z-score < -2 (demand unexpectedly low)
- **Normal:** Within expected range

5.3 Peak/Off-Peak Classification

Binary classification using LightGBM: - **Target:** `is_peak` = 1 if demand $\geq$ 75th percentile, else 0 - **Features:** Same temporal/lag/rolling features - **Model:** LightGBM Classifier (300 trees, `max_depth=6`)

This enables grid operators to proactively identify high-demand periods.

5.4 SHAP Explainability

SHAP (SHapley Additive exPlanations) provides theoretically grounded feature attribution:

- **TreeExplainer:** Exact SHAP values for tree-based models (polynomial complexity, not exponential)
- **Global importance:** Mean $|\text{SHAP}|$ across all samples
- **Local explanations:** Per-prediction feature contributions
- **Dependence plots:** Non-linear relationships between features and predictions

6. Results

6.1 Forecasting Performance

Model	MAE (MW)	RMSE (MW)	R ²	MAPE (%)
XGBoost	1,220	1,520	0.9446	4.41
LightGBM	1,207	1,505	0.9457	4.35

Both models achieve $R^2 > 0.94$, meaning they explain 94%+ of demand variance. LightGBM performs marginally better across all metrics.

Cross-validation (5-fold TimeSeriesSplit): - XGBoost: $R^2 = 0.874 \pm 0.03$ - LightGBM: $R^2 = 0.880 \pm 0.02$

CV scores are lower than single-split scores because each fold has less training data. The low standard deviation confirms stability.

6.2 Residual Analysis

- **Mean residual:** ~ 0 (unbiased predictions)
- **Distribution:** Approximately normal with slight heavy tails
- **ACF:** Significant autocorrelation at lag 24 (daily pattern) — the model captures most but not all daily seasonality
- **Q-Q plot:** Good fit in the center, deviations at tails (extreme values harder to predict)
- **Heteroskedasticity:** Some increasing variance at higher predicted values — typical for load forecasting

6.3 Top SHAP Features

Rank	Feature	SHAP Importance	Interpretation
1	hour_cos	3,549	Time of day (cosine encoding)
2	dayofyear	1,573	Day within the year (seasonality)
3	month	1,414	Month of year
4	is_peak	1,042	Whether demand is in top quartile
5	hour_sin	987	Time of day (sine encoding)

The top features are all temporal — this makes physical sense: electricity demand is driven by human activity patterns (waking, working, sleeping) which repeat daily and seasonally.

6.4 Anomaly Detection Results

Method	Anomalies Detected	Percentage
Isolation Forest	263	1.00%
Z-Score (threshold=3)	~ 50	$\sim 0.19\%$
IQR	~ 30	$\sim 0.11\%$
Spikes (rolling $z > 2$)	$\sim 1,200$	$\sim 4.6\%$
Dips (rolling $z < -2$)	$\sim 1,100$	$\sim 4.2\%$

Method agreement: Isolation Forest is most sensitive (matches its contamination parameter). Z-score and IQR only catch extreme outliers. The multi-method approach provides defense in depth.

6.5 Classification Performance

Metric	Value
AUC-ROC	0.9806
Accuracy	0.954
Precision (peak)	0.91
Recall (peak)	0.89
F1 (peak)	0.90

The classifier achieves **AUC > 0.98**, meaning it almost perfectly distinguishes peak from off-peak hours. This is expected because peak periods follow regular temporal patterns.

7. MLOps & Experiment Tracking

MLflow Integration

- **Backend:** SQLite (portable, no server required)
- **Logged artifacts:** Model weights, metrics, parameters, feature importance
- **Comparison:** Side-by-side model evaluation
- **Reproducibility:** All parameters logged with each run

Project Structure

```
terna_energy_project/  
  src/  
    data/           # Data loading & feature engineering  
    models/         # ML models & explainability  
    monitoring/     # MLflow tracking  
    llm/            # Report generation  
  notebooks/       # 6 Jupyter notebooks (01-06)  
  tests/           # Unit tests  
  figures/         # Diagnostic plots  
  reports/         # Generated reports  
  mlflow_results/  # MLflow SQLite database
```

8. Key Insights & Physical Interpretation

Why $R^2 = 0.94$ Makes Sense

Electricity demand is dominated by deterministic temporal patterns: 1. **Daily cycle:** Morning ramp (6-9am), midday plateau, evening peak (6-9pm), nighttime minimum 2. **Weekly cycle:** Weekday industrial activity vs weekend reduction 3. **Seasonal cycle:** Summer cooling load (AC) vs winter heating load 4. **Annual trend:** Economic growth increases demand ~1-2% per year

Gradient boosting trees can perfectly capture these step-function-like patterns.

Why Residuals Show Lag-24 Autocorrelation

The model uses lag-24 as a feature, which creates a direct copy of the previous day’s value. However, some residual autocorrelation remains because: - The model cannot perfectly predict unusual events (holidays, heat waves) - Weather effects are not included (only temporal features) - Industrial load shifts are not captured

Why Anomaly Detection Works

Power grid anomalies are rare by design (operators maintain stability). The ~1% flagged by Isolation Forest represents genuinely unusual hours — potential data quality issues, extreme weather events, or unusual industrial activity.

9. Limitations & Future Work

Current Limitations

- 1. **No weather features:** Temperature, humidity, and solar irradiance are major demand drivers
- 2. **No holiday calendar:** Italian public holidays create demand anomalies
- 3. **Synthetic data:** Results are validated on synthetic data until ENTSO-E API key is approved
- 4. **No spatial analysis:** Only national aggregate, not regional breakdown
- 5. **No probabilistic forecasts:** Point predictions only, no confidence intervals

Future Improvements

- 1. **Add weather data:** Integrate meteorological features for better accuracy
- 2. **Prophet or N-BEATS:** Compare with specialized time series models
- 3. **Ensemble methods:** Combine XGBoost + LightGBM + linear models
- 4. **Rolling retraining:** Update model weekly with latest data
- 5. **Probabilistic forecasting:** Quantile regression for prediction intervals
- 6. **API deployment:** FastAPI endpoint for real-time predictions

10. Technical Skills Demonstrated

Skill	Implementation
Python	Core language, all modules
SQL	MLflow SQLite backend
Pandas/NumPy	Data manipulation, feature engineering
Scikit-learn	Preprocessing, metrics, model selection
XGBoost	Gradient boosting forecasting
LightGBM	Gradient boosting forecasting + classification
SHAP	Global and local model explainability
MLflow	Experiment tracking, model logging
Matplotlib/Seaborn	Visualization, diagnostic plots
Time Series	Chronological splits, lag features, autocorrelation

Skill	Implementation
Anomaly Detection	Isolation Forest, Z-score, IQR
GenAI/LLM	Report generation (OpenAI integration)
Git	Version control
Testing	Pytest unit tests

11. How to Run

```
# Clone repository
git clone https://github.com/[username]/terna-energy-project.git
cd terna-energy-project

# Create virtual environment
python -m venv .venv
source .venv/bin/activate # Linux/Mac
pip install -r requirements.txt

# Run notebooks in order
jupyter notebook notebooks/01_data_ingestion.ipynb
jupyter notebook notebooks/02_forecasting.ipynb
jupyter notebook notebooks/03_anomaly_detection.ipynb
jupyter notebook notebooks/04_classification.ipynb
jupyter notebook notebooks/05_llm_report.ipynb
jupyter notebook notebooks/06_diagnostics.ipynb

# Run tests
python -m pytest tests/ -v

# View MLflow experiments
mlflow ui --backend-store-uri sqlite:///mlflow_results/mlflow.db
```

12. References

1. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD*.
 2. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*.
 3. Lundberg, S., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*.
 4. ENTSO-E Transparency Platform. <https://transparency.entsoe.eu/>
 5. Terna (Italian TSO). <https://www.terna.it/>
-

This project was developed as a portfolio piece demonstrating end-to-end ML capabilities for the Senior Data Scientist role at Key Partner Digital.